

Aplicación de DevSecOps en el Proyecto Tipo

Integrantes: Iris Carolina Fernández, Juan José Castro, Daniel Díaz

Introducción:

En el contexto actual del desarrollo de software, integrar la seguridad desde las etapas iniciales es fundamental. DevSecOps propone una evolución del enfoque DevOps tradicional al incorporar prácticas de seguridad de forma continua a lo largo del ciclo de vida del desarrollo (S-SDLC). En este documento se describe cómo se podría aplicar DevSecOps a nuestro proyecto tipo, una aplicación web desarrollada en React (frontend), Express.js (backend) y SQLite como base de datos.

Actualmente, el proyecto no cuenta con mecanismos explícitos de seguridad ni automatizaciones asociadas a DevSecOps, por lo que a continuación se plantean propuestas concretas para su adopción, alineadas con las fases del desarrollo y con las buenas prácticas de la industria

Propuesta de integración DevSecOps en la aplicación:

A continuación, se detallan las prácticas DevSecOps que se proponen implementar en el proyecto para mejorar su seguridad y robustez:

- **Planificación con enfoque en seguridad (Shift-Left):** Se propone identificar amenazas comunes desde la fase de requisitos, como inyecciones SQL, exposición de datos sensibles y ataques de denegación de servicio. Esto permitiría definir requisitos no funcionales de seguridad desde el inicio.
- **Diseño seguro de arquitectura:** Para prevenir accesos no autorizados, se plantea aplicar el principio de menor privilegio, segmentando el backend, la base de datos y los servicios de autenticación en módulos independientes. Se sugiere implementar control de acceso basado en roles (RBAC).
- **Codificación segura:** Se recomienda introducir validaciones de entrada en los endpoints de Express, así como sanitizar cualquier dato que se renderice en el frontend. También se sugiere gestionar las credenciales y variables sensibles usando archivos .env y evitar hardcodearlas en el código fuente.
- **Automatización de análisis de seguridad:** Se propone configurar herramientas como ESLint y Prettier en todo el proyecto, además de integrar un análisis estático del código con herramientas como SonarCloud o CodeQL. Esto permitiría detectar malas prácticas o vulnerabilidades antes del despliegue.

- **Integración y entrega continua (CI/CD):** Aunque actualmente no se cuenta con pipelines, se propone crear flujos de integración continua con GitHub Actions. Estos pipelines podrían ejecutar pruebas automáticas, análisis de seguridad y validaciones antes del despliegue.
- **Testing de seguridad y dependencias:** Se sugiere incluir herramientas como npm audit, Dependabot o Snyk para detectar vulnerabilidades en las librerías utilizadas en frontend y backend.
- **Despliegue seguro y monitorización:** Finalmente, se recomienda automatizar el despliegue en entornos controlados y con control de versiones. La monitorización del backend (con herramientas como Prometheus o PM2) permitiría detectar comportamientos anómalos tras el despliegue.

DevSecOps dentro del ciclo S-SDLC:

En la tabla se detalla cómo se propone aplicar DevSecOps en cada fase del ciclo de vida del desarrollo seguro (S-SDLC):

Fase del S-SDLC	Propuesta de aplicación DevSecOps
Requisitos	Identificación de amenazas y definición de requisitos de seguridad desde la planificación.
Diseño	Aplicar diseño modular seguro, principio de menor privilegio y análisis de amenazas.
Implementación	Uso de linters, validación y sanitización de entradas, gestión segura de credenciales.
Verificación	Inclusión de pruebas automatizadas, análisis estático y escaneo de dependencias.
Despliegue	Automatización mediante CI/CD con validaciones de seguridad antes de producción.
Mantenimiento	Monitorización de logs, gestión de actualizaciones y control de acceso post-despliegue.